NumPy Essentials: Arrays, Indexing, Broadcasting, and Performance

Prof. Wook-Shin Han

Spring 2026

Roadmap

1. NumPy at a Glance
2. Creation: Semantics
3. Operations: Semantics
4. Arrays and DTypes
5. Indexing and Shaping
6. Math and Linear Algebra
7. Broadcasting
8. Vectorization and Performance
9. Case Studies
10. Debugging and Best Practices

Why NumPy

- N-dimensional arrays with *contiguous* memory for fast *vectorized* computation[1].
- Rich library of universal functions, reductions, and linear algebra.
- Broadcasting rules enable concise expressions over compatible shapes.
- Serves as a foundation for scientific Python stacks.

[1] Vectorized computation means performing operations on entire arrays at once (e.g., adding two arrays with `c = a + b`), instead of using slow Python loops to process each element individually. This leverages optimized low-level code (in C/Fortran) and CPU features like SIMD for significant speedups, especially on large datasets.

Example: Why NumPy (Vectorization Glimpse)

```
import numpy as np, time

rng = np.random.default_rng(0)
x = rng.random(400_000)
y = rng.random(400_000)

t0 = time.perf_counter()
s_loop = 0.0
for i in range(x.size):
    s_loop += x[i] * y[i]
t1 = time.perf_counter()

t2 = time.perf_counter()
s_vec = (x * y).sum()
t3 = time.perf_counter()

print("loop:", t1 - t0, "sec")
```

Intuition: What is a NumPy Array?

```
import numpy as np
A = np.array([[ 1,  2,  3,  4],
              [ 5,  6,  7,  8],
              [ 9, 10, 11, 12]], dtype=np.int32)

print(A.shape)   # (3, 4)
print(A[1, 2])   # 7  (row index 1, column index 2)
```

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}, \qquad \mathrm{shape}(A) = (3, 4)$$

- **Shape** $(3, 4)$: 3 rows (axis 0 length $= 3$), 4 columns (axis 1 length $= 4$).
- **Indexing**: choose a cell by (i, j). With 0-based indices here, $i \in \{0, 1, 2\}$, $j \in \{0, 1, 2, 3\}$; e.g., $A[1, 2] = 7$.
- **Dtype**: all entries share one type (here `int32`).

NumPy Array: metadata vs. elements (1/2)

Fixed parameters (metadata)

- Rank (number of axes): $n \in \mathbb{N}$.
- Shape: $s = (s_1, \ldots, s_n)$ is the tuple of axis lengths; here s_k denotes the length along axis k ($s_k \in \mathbb{N}$, $s_k \geq 1$).
- Data type (dtype): τ with value domain $\mathcal{V}_\tau$ (e.g., `int64` and `float64`).

Index set and semantics

- Index set: $I_s = \{\, i = (i_1, \ldots, i_n) \in \mathbb{Z}^n \mid 0 \leq i_k < s_k \ \forall k \,\}$.
- Universe of arrays (shape s, dtype τ): $\mathcal{V}_\tau^{I_s} = \prod_{i \in I_s} \mathcal{V}_\tau$.
- A particular array instance: $A \in \mathcal{V}_\tau^{I_s}$ (equivalently, $A : I_s \to \mathcal{V}_\tau$).
- Element values: for $i \in I_s$, $A[i] \in \mathcal{V}_\tau$.

NumPy Array: concrete memory realization (2/2)

Concrete memory realization

- Realized by (s, τ, σ, p), where $\sigma = (\sigma_1, \ldots, \sigma_n) \in \mathbb{Z}^n$ are *byte strides*[2] and p is the address of $A[0, \ldots, 0]$.
- Address of $A[i]$ for $i = (i_1, \ldots, i_n) \in I_s$: $\quad \mathrm{addr}(i) = p + \sum_{k=1}^{n} i_k \, \sigma_k$.
- Address-computation cost: $O(1)$.

[2] Formally: for the standard basis vector e_k, "advance along axis k" means replace i by $i + e_k$ (increment i_k by $+1$ while keeping other coordinates fixed). Then $\sigma_k = \mathrm{addr}(i + e_k) - \mathrm{addr}(i)$.

Verifying the Formal Definition (1/2)

```
X  = np.arange(12, dtype=np.int32).reshape(3, 4, order='C')
XF = np.asfortranarray(X)

print("shape:", X.shape, "ndim:", X.ndim, "size:", X.size)
# shape: (3, 4) ndim: 2 size: 12

print("dtype:", X.dtype, "itemsize:", X.itemsize)
# dtype: int32 itemsize: 4   (bytes per element)

print("strides (C):", X.strides, "C-contig?", X.flags['C_CONTIGUOUS'])
# strides (C): (16, 4) C-contig? True
#   (row-major: to move down one row: 4*4=16 bytes, to move along one col: 4 bytes)

print("strides (F):", XF.strides, "F-contig?", XF.flags['F_CONTIGUOUS'])
# strides (F): (4, 12) F-contig? True
#   (col-major: to move down one row: 4 bytes, to move along one col: 3*4=12 bytes)

prod = 1
for s in X.shape:
    prod *= s
print("product(shape) == size ?", prod == X.size)
# product(shape) == size ? True   (3*4 = 12)
```

Verifying the Formal Definition (2/2)

```
X = np.arange(12, dtype=np.int32).reshape(3, 4, order='C')
# shape=(3, 4), dtype=int32, itemsize=4, strides=(16, 4) in bytes

V = X[::2, ::2]   # view (strided slice: rows 0,2 and cols 0,2)
# V =[[ 0  2]
#  [ 8 10]]

C = X.copy()      # deep copy (new buffer)

print("X.base is None?", X.base is None)
# X.base is None? False          # X is a view, not the original memory owner

print("V.base is X?", V.base is X)
# V.base is X? False
# A view's base always points to the original
# memory buffer, not an intermediate view

print("C.base is None?", C.base is None)
# C.base is None? True          # copy owns its memory (no base)

data = X.ctypes.data
i, j = 2, 1
expected = data + i * X.strides[0] + j * X.strides[1]
addr_ij = X[i:i+1, j:j+1].ctypes.data
print("address formula holds?", addr_ij == expected)
# address formula holds? True   # pointer arithmetic matches (i,j) element

XT = X.T
print("X.T shape:", XT.shape, "strides:", XT.strides)
# X.T shape: (4, 3) strides: (4, 16)   # axes swapped  strides swapped
```

Creating arrays: intuition & quick demo

```
import numpy as np

a = np.array([[1, 2], [3, 4]], dtype=np.int16)   # build from lists (copy)
b_src  = np.arange(6, dtype=np.float32).reshape(2, 3)
b_view = np.asarray(b_src)                        # may be a view (no copy)
b_copy = np.array(b_src)                          # forced copy

z = np.zeros((2, 3), dtype=np.int32)              # fill with 0s
o = np.ones((2, 3), dtype=np.int32)               # fill with 1s
f = np.full((2, 3), 7, dtype=np.int32)            # fill with constant
e = np.empty((2, 3), dtype=np.int32)              # uninitialized data

print("a:", a.shape, a.dtype)
print("b_view shares base?", b_view.base is b_src)
print("b_copy shares base?", b_copy.base is b_src)
print("z/o/f/e shapes:", z.shape, o.shape, f.shape, e.shape)
```

- **np.array**: constructs a new array (copy); **np.asarray**: reuse memory when safe (view).
- **zeros/ones/full**: allocate then fill; **empty**: allocate without initializing values.
- **Shape** is axis lengths; **dtype** is the common scalar type stored.

Creation routines: mathematical semantics

Let $s \in \mathbb{N}^n$ and dtype τ with value set $\mathcal{V}_\tau$.

- **Array from sequence** (`np.array`, `np.asarray`): parse a finite nested sequence into shape s; entries map to $\mathcal{V}_\tau$ via promotion/conversion. `np.array` returns a copy; `np.asarray` may return a view when τ and layout match.
- **Zeros/Ones/Full:**

$$\begin{aligned} \texttt{zeros}(s): &\ X[i] = 0_\tau, \\ \texttt{ones}(s): &\ X[i] = 1_\tau, \\ \texttt{full}(s, c): &\ X[i] = c \in \mathcal{V}_\tau \quad (\forall i \in I_s). \end{aligned}$$

- **Empty:** `empty`(s) returns X with unspecified $X[i]$ (arbitrary buffer contents).

Reshape essentials: views, −1, and order

```
import numpy as np

x = np.arange(12)                    # shape (12,)

A = x.reshape(3, 4)                  # (3, 4) view when possible
B = x.reshape(-1, 6)                 # infer one dim -> (2, 6)
C = x.reshape(3, 4, order='F')       # same data, different strides
D = A.ravel()                        # flatten view when possible
E = A.flatten()                      # flatten copy

print(A.shape, B.shape, C.shape) # (3, 4) (2, 6) (3, 4)
print("A shares base?", A.base is x)   # usually True
print("flatten is copy?", E.base is None)
```

- `reshape` preserves element order and count; often returns a *view* (no copy).
- Use one `-1` to let NumPy infer that dimension.
- `order='C'/'F'` chooses index traversal; may change strides (still a view if possible).
- `ravel`: flattened *view* when possible; `flatten`: always a *copy*.

Adding a new axis: three idioms (row/column/batch)

```
v = np.arange(4)                    # shape (4,)
# Row vector (1, 4)
v_row1 = v[None, :]                 # None is alias of np.newaxis
v_row2 = v[np.newaxis, :]
v_row3 = np.expand_dims(v, axis=0)
v_row4 = v.reshape(1, -1)

# Column vector (4, 1)
v_col1 = v[:, None]
v_col2 = np.expand_dims(v, axis=1)
v_col3 = v.reshape(-1, 1)

# Insert axes at front / middle / end for broadcasting
M = np.arange(6).reshape(2, 3)    # (2, 3)
M0 = M[None, :, :]                  # (1, 2, 3)   batch axis
M1 = M[:, None, :]                  # (2, 1, 3)   pairwise-prep
M2 = M[:, :, None]                  # (2, 3, 1)   channel axis

print(v_row1.shape, v_col1.shape, M0.shape, M1.shape, M2.shape)
```

- `None`/`np.newaxis` and `np.expand_dims` are equivalent for inserting size-1 axes.
- `reshape(..., 1, ...)` also works

Squeeze, keepdims, and moving axes

```
X = np.arange(24).reshape(2, 3, 4)

# Reductions: keep singleton axes for safe broadcasting later
r = X.sum(axis=1, keepdims=True)   # shape (2, 1, 4)

# Remove size-1 axes explicitly
r_squeezed = np.squeeze(r, axis=1) # shape (2, 4)

# Rearranging axes
T1 = np.swapaxes(X, 0, 1)          # (3, 2, 4)
T2 = np.moveaxis(X, 2, 0)          # (4, 2, 3)
T3 = X.transpose(1, 0, 2)          # (3, 2, 4)

# Quick promotion helpers
a2 = np.atleast_2d(np.arange(3))   # (1, 3)
a3 = np.atleast_3d(np.arange(3))   # (1, 3, 1)
```

- `keepdims=True` preserves reduced axes as size 1 (often handy before broadcasting).
- `squeeze` removes axes of length 1 when you no longer need them.
- `swapaxes`, `moveaxis`, `transpose` reorder axes explicitly.

Intuition: Grid constructors (why grid?)

```
# 1D coordinate generators
r = np.arange(2, 10, 3)            # [2, 5, 8]  -> step on a half-open interval
l = np.linspace(0.0, 1.0, 5)       # 5 evenly spaced points in [0, 1]

# Turn 1D coordinates into a 2D Cartesian grid via broadcasting
xs = np.arange(4)                  # [0, 1, 2, 3]
ys = np.arange(3)                  # [0, 1, 2]
X = xs[np.newaxis, :]              # shape (1, 4)
Y = ys[:, np.newaxis]              # shape (3, 1)
G = X + 10*Y                       # shape (3, 4) labels each grid row

# Canonical 2D patterns
E = np.eye(3, k=1)                 # ones on the first superdiagonal
D = np.diag([9, 8, 7], k=-1)       # place vector on the subdiagonal
```

- **Why grid?** `arange`/`linspace` create 1D *coordinate lines*; combining them (via broadcasting or `meshgrid`) yields a *Cartesian grid* of points $(x_i, y_j, \dots)$.
- `arange` walks by a fixed step; `linspace` splits an interval into equal fractions.
- `eye`/`diag` build structured 2D grids with ones or a vector along a chosen diagonal.

Grid Constructors: `arange`, `linspace`, `eye`, `diag`

- `np.arange(a,b,h,dtype)` (with $h \neq 0$): defines length

$$N = \max\left\{ n \in \mathbb{N}_0 : a + nh \in \begin{cases} [a, b) & \text{if } h > 0, \\ (b, a] & \text{if } h < 0 \end{cases} \right\},$$

  and returns $x \in \mathcal{V}_\tau^{\{0,\ldots,N-1\}}$ with $x_i = a + ih$. For integer τ the mapping is exact; for floating τ rounding may occur.

- `np.linspace(a,b,num=N,endpoint=True)`:

$$x_i = \begin{cases} a + \dfrac{i}{N-1}(b-a), & \text{if endpoint is True and } N \geq 2, \\ a + \dfrac{i}{N}(b-a), & \text{if endpoint is False and } N \geq 1. \end{cases}$$

Vector Indexing (Slicing): Intuition

- In NumPy, a 1D slice `a[i:j:k]` selects elements on a *half-open* arithmetic progression.
- **Half-open:** start at index i, step by k, **stop before** j.
- Works with negative indices (wrap from end) and negative steps (reverse walks).
- Slices are *views* (no copy) when possible; changing the slice can change the original array.

Slice Syntax & Defaults

```
import numpy as np
a = np.arange(10)          # [0 1 2 3 4 5 6 7 8 9]

print(a[2:7])              # [2 3 4 5 6]  (i=2, j=7, k=1)
print(a[:5])               # [0 1 2 3 4]  (i defaults to 0)
print(a[5:])               # [5 6 7 8 9]  (j defaults to len(a))
print(a[::2])              # [0 2 4 6 8]  (k=2)
print(a[::-1])             # [9 8 7 6 5 4 3 2 1 0] (reverse)
```

- **Defaults:** `i` defaults to 0, `j` to `len(a)`, `k` to 1.
- **Negative indices:** `-1` is the last element, `-2` the second last, etc.

Half-Open Interval: Quick Check

```
a = np.arange(10)
s = a[2:7]      # expect len = 5 -> indices 2,3,4,5,6
print(len(s), s)   # 5 [2 3 4 5 6]

# contrast with Python range
print(list(range(2, 7)))  # [2, 3, 4, 5, 6]  (same half-open idea)
```

- Length for $k > 0$: $\max\{0, \lceil (j-i)/k \rceil$ with half-open stop.
- For $k < 0$: indices go $i, i+k, i+2k, \ldots$ while $> j$, still half-open.

Views vs Copies (1D Slices are Views)

```
a = np.arange(6)
v = a[1:5]      # view of a's buffer
v[0] = 999
print(a)        # [  0 999   2   3   4   5]

# Advanced indexing (fancy) is a copy:
idx = [1, 2, 3]
c = a[idx]      # copy
c[0] = -1
print(a)        # unchanged at positions 1,2,3
```

- **Basic slicing** (colon syntax) ⇒ usually a view.
- **Advanced indexing** (lists/arrays/bool masks) ⇒ a copy.

Slice Assignment

```
a = np.arange(10)
a[2:7] = −1          # broadcast scalar across the slice
print(a)             # [0 1 −1 −1 −1 −1 −1 7 8 9]

a = np.arange(10)
a[::2] = 100         # assign to every other element
print(a)             # [100 1 100 3 100 5 100 7 100 9]

a = np.arange(6)
a[0:3] = [9, 8, 7]   # length must match the slice length
print(a)             # [9 8 7 3 4 5]
```

- Right-hand side can be a scalar (broadcast) or an array matching the slice length.

Negative Indices & Negative Steps

```
a = np.arange(10)
print(a[-3:])      # last 3 -> [7 8 9]
print(a[:-3])      # all but last 3 -> [0 1 2 3 4 5 6]

# walking backwards (k < 0)
print(a[8:3:-2])   # indices: 8,6,4 -> [8 6 4]
print(a[::-3])     # [9 6 3 0]
```

- With $k < 0$, the walk goes right-to-left; the stop index is still half-open.

Slicing and Strides (Performance Intuition)

```
X = np.arange(12).reshape(3, 4)
row = X[1, :]          # contiguous view of a row
col = X[:, 2]          # strided view of a column

print("row:", row, "row.strides:", row.strides)
print("col:", col, "col.strides:", col.strides)
```

- Slices change *strides* (step in bytes between consecutive elements along an axis).
- Row slices are often contiguous; column slices in C-order are strided.

Common 1D Slicing Patterns

```
a = np.arange(10)
first3   = a[:3]        # [0 1 2]
last3    = a[-3:]       # [7 8 9]
middle   = a[3:7]       # [3 4 5 6]
evens    = a[::2]       # [0 2 4 6 8]
odds     = a[1::2]      # [1 3 5 7 9]
reverse  = a[::-1]      # [9 8 7 6 5 4 3 2 1 0]
```

- Combine start/stop/step for windows, skipping, and reversing without loops.

Intuition: Random numbers

```
import numpy as np
G1 = np.random.default_rng(42)
G2 = np.random.default_rng(42)

X = G1.normal(loc=0.0, scale=1.0, size=(2, 3))
Y = G2.normal(loc=0.0, scale=1.0, size=(2, 3))

print(X.shape, Y.shape)          # (2, 3) (2, 3)
print(np.allclose(X, Y))         # True (same seed -> same draws)
```

- A **generator** with a seed produces reproducible arrays of random draws.
- Each method chooses a **distribution** (e.g., normal, uniform) and a **shape**.

Random Generation: Distribution Semantics (revised)

Let $G =$ `np.random.default_rng(seed)`.

- `G.random(s)` returns $X \in [0,1)^{l_s}$ with $X[i]$ i.i.d. $\sim \mathrm{Unif}(0,1)$ (continuous).
- Other methods specify distributions pointwise:
  - `G.normal`(μ, σ, s): $X[i] \sim \mathcal{N}(\mu, \sigma^2)$ i.i.d.
  - `G.uniform`(ℓ, h, s): $X[i] \sim \mathrm{Unif}(\ell, h)$ (continuous) i.i.d.
  - `G.integers`$(a, b, s,$ `endpoint=False`$)$: $X[i]$ i.i.d. $\sim \mathrm{DiscreteUniform}\big(\{a, a+1, \ldots, b-1\}\big)$, i.e.

    $$\Pr\{X[i] = k\} = \tfrac{1}{b-a} \quad \text{for } k \in \{a, \ldots, b-1\}.$$

    With `endpoint=True`, the support is $\{a, \ldots, b\}$ and $\Pr\{X[i] = k\} = 1/(b-a+1)$.
- Every element $X[i]$ is an independent draw

Example: Random Generation and Reproducibility

```
import numpy as np

G1 = np.random.default_rng(42)
G2 = np.random.default_rng(42)

X = G1.normal(loc=0.0, scale=1.0, size=(100000,))
Y = G2.normal(loc=0.0, scale=1.0, size=(100000,))

print("same draws?", np.allclose(X, Y))
print("mean, std ~ 0,1:", X.mean(), X.std())
```

Visualizing `np.eye`: identity & shifted diagonals

$$\texttt{np.eye(3)} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\texttt{np.eye(4,k=-1)} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$\texttt{np.eye(2,4,k=1)} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

- $k = 0$: main diagonal; $k > 0$: superdiagonal; $k < 0$: subdiagonal.
- Rectangular shapes allowed: `np.eye(N, M, k)`.

`np.eye`: identity & shifted identity

- **Signature**: `np.eye(N, M=None, k=0, dtype=float)`.
- Returns $E \in \mathcal{V}_\tau^{N \times M}$ (with $M = N$ if omitted) s.t.

$$E_{ij} = \begin{cases} 1_\tau, & \text{if } j - i = k, \\ 0_\tau, & \text{otherwise.} \end{cases}$$

  Rectangular shapes and offsets k are supported; out-of-range diagonals are zeros.
- Typical uses: identity for linear algebra, regularization (λI), masks, stencils.

What is the k-th diagonal? (NumPy, 0-based)

Given $A \in \mathbb{R}^{N \times M}$, the k-th diagonal is

$$\{\, A_{i,j} \mid j - i = k,\ 0 \le i < N,\ 0 \le j < M \,\}.$$

$$A = \begin{bmatrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} & a_{0,4} \\ a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} & a_{1,4} \\ a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} & a_{2,4} \\ a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} & a_{3,4} \end{bmatrix}$$

$$k = 0\colon\ (a_{0,0},\ a_{1,1},\ a_{2,2},\ a_{3,3})$$
$$k = 1\colon\ (a_{0,1},\ a_{1,2},\ a_{2,3},\ a_{3,4})$$
$$k = -1\colon\ (a_{1,0},\ a_{2,1},\ a_{3,2})$$

Length. For A of size $N \times M$,

$$L = \max\Big(0,\ \min\big(N - \max(0, -k),\ M - \max(0, k)\big)\Big),$$

and for square $n \times n$, this simplifies to $L = \max(0,\ n - |k|)$.

Visualizing `np.diag`: construct vs. extract

Construct from a vector

$$v = \begin{bmatrix} 9 \\ 8 \\ 7 \end{bmatrix}, \quad \texttt{np.diag(v)} = \begin{bmatrix} 9 & 0 & 0 \\ 0 & 8 & 0 \\ 0 & 0 & 7 \end{bmatrix}$$

$$\texttt{np.diag(v, k=1)} = \begin{bmatrix} 0 & 9 & 0 & 0 \\ 0 & 0 & 8 & 0 \\ 0 & 0 & 0 & 7 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$k > 0$ places v on a superdiagonal; size grows to fit.

Extract from a matrix

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

$$\texttt{np.diag(A)} = \begin{bmatrix} 1 \\ 5 \\ 9 \end{bmatrix}$$

$$\texttt{np.diag(A, k=-1)} = \begin{bmatrix} 4 \\ 8 \end{bmatrix}$$

$k < 0$ extracts a subdiagonal.

- Vector input $\Rightarrow$ *construct* a (possibly shifted) diagonal matrix.
- Matrix input $\Rightarrow$ *extract* a (possibly shifted) diagonal as a 1D array.

`np.diag`: construct or extract a diagonal

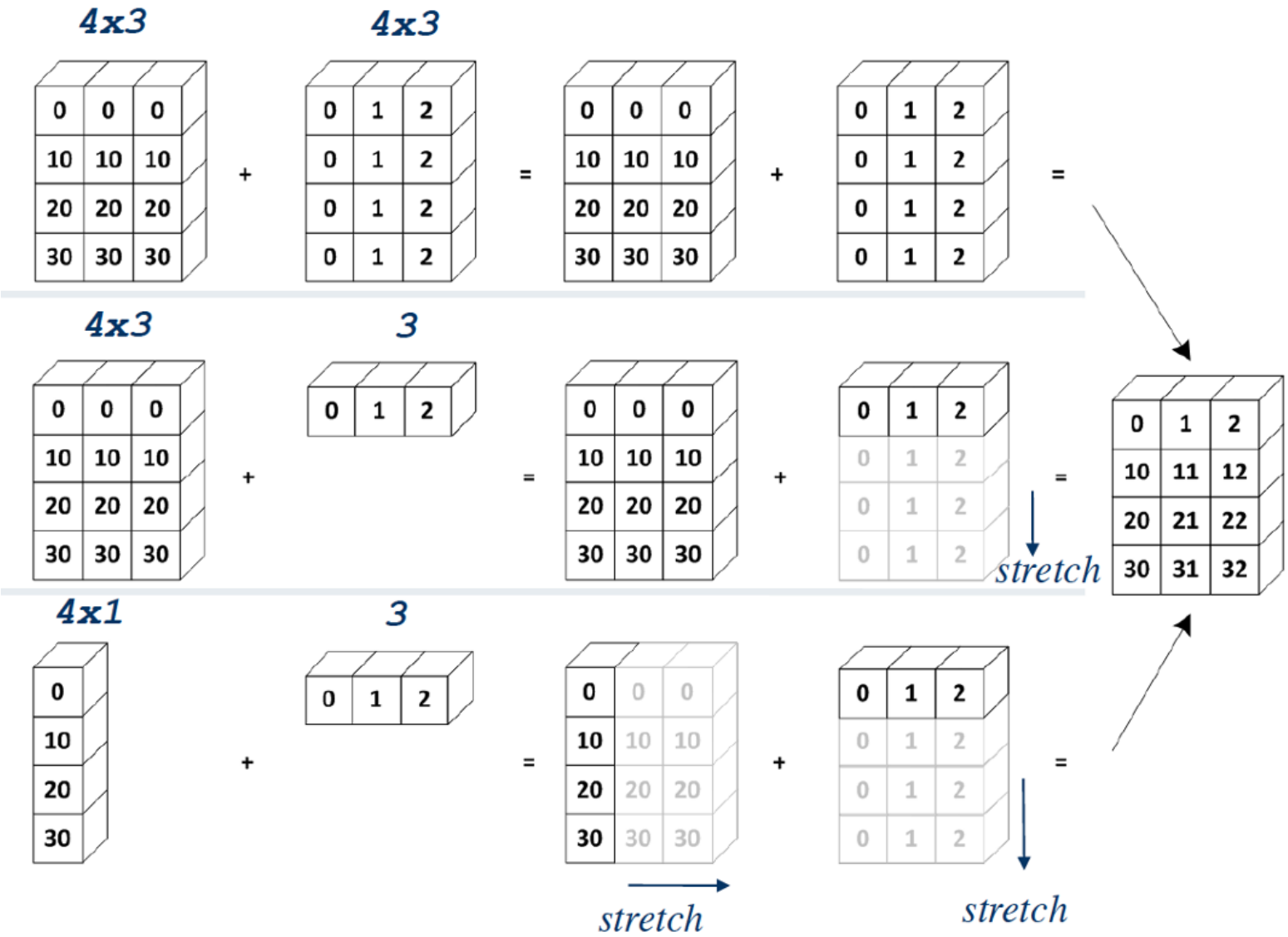
- **Vector input (construct)**: for v of length L, `np.diag(v, k)` returns a 2D array D with
$$D_{i,\,i+k} = v_i, \quad \text{zeros elsewhere,}$$
and **output shape** $(L + |k|) \times (L + |k|)$.
- **Matrix input (extract)**: for $A \in \mathbb{R}^{N \times M}$, `np.diag(A, k)` returns the k-th diagonal as a 1D array with length
$$\begin{cases} \min(N,\, M - k), & k \geq 0, \\ \min(N + k,\, M), & k < 0, \end{cases}$$
taking all valid $A_{i,\,i+k}$.
- Related: `np.diagonal` (ND diagonals), `np.fill_diagonal` (write into main diagonal).

Intuition: Broadcasting

```
import numpy as np
A = np.arange(6).reshape(2, 3)    # shape (2, 3)
b = np.array([10, 20, 30])        # shape (3,)
c = np.array([[100], [200]])      # shape (2, 1)

Y1 = A + b    # (2, 3) + (3,)   -> (2, 3)
Y2 = A + c    # (2, 3) + (2, 1) -> (2, 3)

print(Y1.shape, Y2.shape)         # (2, 3) (2, 3)
print(Y1[0], Y2[0])               # first rows for a quick check
```

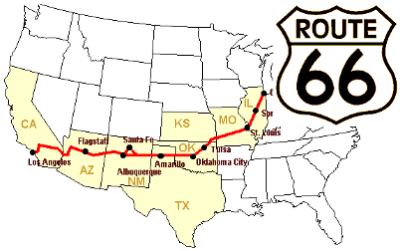
- Dimensions of size 1 are **repeated** to match the other operand.
- Shapes align from the **right**; no data are copied just to match sizes.

Broadcasting (visual overview)



Application: Distances between cities on Route 66

- Given a 1D array of *mileposts* (cumulative distance from Chicago) for cities on Route 66:

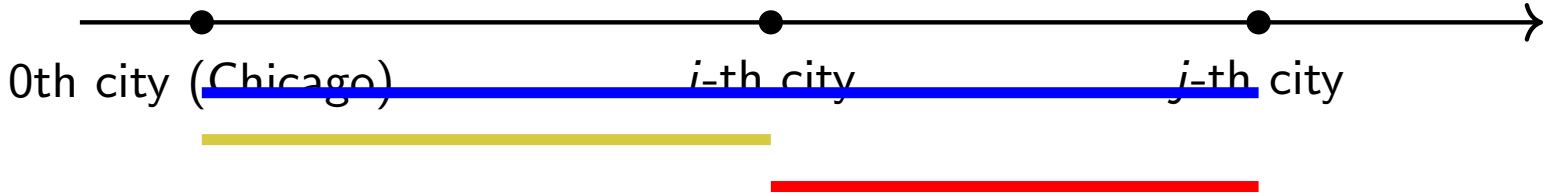


$$\texttt{mileposts} = [0,\ 198,\ 303,\ 736,\ 871,\ 1175,\ 1475,\ 1544,\ 1913,\ 2448]$$

- Goal: construct a 2D array of pairwise city distances.

From mileposts to pairwise distance (1D road model)

$$d(i,j) \;=\; \left|\,\text{milepost}(j) \;-\; \text{milepost}(i)\,\right|$$



On a single road, the distance between cities equals the absolute difference of their mileposts.

Broadcasting solution in NumPy

```
import numpy as np

mileposts = np.array([0, 198, 303, 736, 871, 1175, 1475, 1544, 1913, 2448])

# Broadcasting trick: (N,) and (N,1) -> (N,N)
distance_array = np.abs(mileposts - mileposts[:, np.newaxis])

print(distance_array)
# Each (i,j) entry is |mileposts[j] - mileposts[i]|
```

Why this works: `mileposts` has shape `(N,)`, `mileposts[:, np.newaxis]` has shape `(N,1)`. Broadcasting produces a `(N,N)` matrix of pairwise differences.

Broadcasting for pairwise difference

Broadcasting (two arrays): shapes (1/2)

Let X have shape $s = (s_1, \ldots, s_n)$ and Z have shape $t = (t_1, \ldots, t_m)$. We apply an elementwise binary function g (e.g. `np.add`, `np.maximum`).

Step A: Right-align & check compatibility.

- Let $r = \max(n, m)$. Build aligned shapes $\tilde{s}, \tilde{t} \in \mathbb{N}^r$ by *prefixing 1s* on the shorter one.
- Axis j is *compatible* iff $\tilde{s}_j = \tilde{t}_j$ or one of them is 1.
- If any axis is incompatible, NumPy raises `ValueError`.

Step B: Broadcasted result shape.

$$u = \text{broadcast}(s, t) \;:=\; (\max(\tilde{s}_1, \tilde{t}_1), \ldots, \max(\tilde{s}_r, \tilde{t}_r)).$$

Broadcasting (two arrays): shapes (2/2)

Assume you already aligned shapes to $\tilde{s}, \tilde{t}$ and found the result shape u.

Rule for picking elements

- Look at one output position $i = (i_1, \ldots, i_r)$.
- For each axis j:
  - If X has size 1 on axis j ($\tilde{s}_j = 1$), use index 0 for X on that axis; otherwise use i_j.
  - If Z has size 1 on axis j ($\tilde{t}_j = 1$), use index 0 for Z on that axis; otherwise use i_j.
- Then compute
  $Y[i] = g(X[\text{picked indices for } X], Z[\text{picked indices for } Z])$.

Why? Here $\tilde{s} = (2, 3)$ and $\tilde{t} = (1, 3)$. Since $\tilde{t}_0 = 1$, Z ignores i_0 and always uses 0 on axis 0:

$$Z_{\text{broadcast}} = \begin{bmatrix} z_0 & z_1 & z_2 \\ z_0 & z_1 & z_2 \end{bmatrix}, \qquad Y = X + Z_{\text{broadcast}}.$$

Hence $Y[i_0, i_1] = X[i_0, i_1] + Z[0, i_1]$.

Broadcasting (many arrays): informal recipe

Goal: apply a pointwise function u to $X^{(1)}, \ldots, X^{(m)}$ of possibly different shapes.
Recipe

- **Right-align shapes:** pad missing *leading* axes with 1s so all have the same number of axes.
- **Axis check (one axis at a time):** ignoring 1s, all remaining sizes on that axis must be the *same*. If you see two different non-1 sizes on the same axis, broadcasting *fails*.
- **Result shape:** per axis, take the *maximum* size you saw on that axis.
- **How values are used:** any input with size 1 on an axis is conceptually *reused* (stretched) along that axis; if its size equals the result size, indices pass through unchanged.
- **Scalars:** behave like shape `()` and broadcast to anything.

Memory note. Broadcasting itself does not copy data along size-1 axes; the *output* array of an operation is newly allocated (unless you write into `out=`).

Broadcasting (many arrays): quick shape examples

Compatible (values stretch on size 1 axes)

A	:	(2, 1, 3)
B	:	(1, 4, 1)
C	:	(2, 4, 1)
result	:	(2, 4, 3)

Here: A stretches on axis 1, B stretches on axes 0 and 2, C stretches on axis 2.

With a scalar and a vector

M	:	(7, 1)
v	:	(1,)
c	:	() (scalar)
result	:	(7, 1)

v stretches along both axes; c stretches everywhere.

Incompatible (two different non-1 sizes on the same axis)

X	:	(2, 3)
Y	:	(3, 1)
Z	:	(2, 2)
status	:	error at last axis: sizes $\{3, 1, 2\}$ $\Rightarrow$ non-1 sizes 3 and 2 conflict

Example: Broadcasting Index Semantics

```
import numpy as np

A = np.arange(6).reshape(2, 3)
b = np.array([10, 20, 30])        # shape (3,)
c = np.array([[100], [200]])      # shape (2,1)

Y1 = A + b          # (2,3) + (3,) -> (2,3)
Y2 = A + c          # (2,3) + (2,1) -> (2,3)
Y3 = b + c          # (3,) + (2,1) -> (2,3)

print("shapes:", Y1.shape, Y2.shape, Y3.shape)
print(np.allclose(Y1, Y2))        # different broadcasting but same shape
```

Intuition: Elementwise, pairwise, reductions

```
x = np.array([1., 2., 3.])        # shape (3,)
y = np.array([10., 20., 30., 40.])  # shape (4,)

E = x * 2.0                       # elementwise -> (3,)
P = np.add.outer(x, y)            # pairwise (outer) -> (3, 4)

M = np.arange(12).reshape(3, 4) # shape (3, 4)
r0 = M.sum(axis=0)                # reduce rows -> (4,)
r1 = M.sum(axis=1)                # reduce cols -> (3,)

print(E.shape, P.shape, r0.shape, r1.shape)
```

- **Elementwise**: operate entry-by-entry (with broadcasting).
- **Pairwise (outer)**: combine every x_i with every y_j.
- **Reduction**: collapse an axis with a combiner (sum, max, ...).

Elementwise vs Pairwise (Outer)

- **Elementwise (pointwise)**: a unary or binary u applied under broadcasting, e.g. $Y = u(X)$ or $Y = u(X, Z)$. After shape alignment, each output index i depends on at most one index from each operand.
- **Pairwise (outer)**: for vectors $x \in \mathcal{V}_\alpha^{\{0,\ldots,M-1\}}$, $y \in \mathcal{V}_\beta^{\{0,\ldots,N-1\}}$ and a bivariate f,

$$F \in \mathcal{V}_\gamma^{M \times N}, \qquad F_{ij} = f(x_i, y_j).$$

  In NumPy this is captured by *outer* semantics (e.g. `np.subtract.outer`, `np.multiply.outer`) or by broadcasting with inserted axes.

Splitting indices: i_A and $i_{\bar{A}}$ (and NumPy mapping)

Keep X with shape $s = (2, 3, 4)$ and pick the reduced axis set $A = \{2\}$ (i.e., axis 2 in math, axis 1 in NumPy). Then $\bar{A} = \{1, 3\}$.

Index split

$$i = (i_1, i_2, i_3) \in I_{\{1,2,3\}} \quad \Rightarrow \quad i_A = (i_2) \in I_{\{2\}}, \quad i_{\bar{A}} = (i_1, i_3) \in I_{\{1,3\}}.$$

Reduction meaning

$$\big(\mathrm{reduce}_{A,+} X\big)[i_{\bar{A}}] \;=\; \sum_{i_2=0}^{s_2-1} X[i_1,\ i_2,\ i_3].$$

Example: at $(i_1, i_3) = (1, 2)$,

$$\big(\mathrm{reduce}_{\{2\},+} X\big)[1, 2] \;=\; X[1, 0, 2] + X[1, 1, 2] + X[1, 2, 2].$$

NumPy code (axis indices are 0-based)

```
import numpy as np
X = np.arange(24).reshape(2, 3, 4)  # shape (2,3,4)
R = np.sum(X, axis=1)               # reduce along math-axis 2 -> numpy axis=1
# Check a single entry equals the explicit sum above:
i1, i3 = 1, 2
assert R[i1, i3] == X[i1, 0, i3] + X[i1, 1, i3] + X[i1, 2, i3]
```

Reductions: symbols, meaning, and shape

Setting. Let X be an n-D array with shape $s = (s_1, \ldots, s_n)$.

Symbols used

- $\oplus$: an *associative* binary operator with identity e (e.g., $(+, 0)$, $(\max, -\infty)$, $(\wedge, \text{True})$).
- $A \subseteq \{1, \ldots, n\}$: set of *axes to reduce*. Its complement is $\bar{A} = \{1, \ldots, n\} \setminus A$ (axes we *keep*).
- For any axis set J, the index set $I_J = \prod_{k \in J} \{0, \ldots, s_k - 1\}$ (Cartesian product over those axes). Indices are split as $i_A \in I_A$ and $i_{\bar{A}} \in I_{\bar{A}}$.

Definition (mathematical)

$$\left(\mathrm{reduce}_{A,\oplus} X\right)[\, i_{\bar{A}} \,] \;=\; \bigoplus_{i_A \in I_A} X[\, i_A,\, i_{\bar{A}} \,].$$

The result has shape $\left(s_k\right)_{k \in \bar{A}}$. Option `keepdims` retains each reduced axis $k \in A$ with size 1.

NumPy mapping

$\mathrm{reduce}_{A,+} \leftrightarrow$ `np.sum(X, axis=...)`, $\quad \mathrm{reduce}_{A,\max} \leftrightarrow$ `np.max(X, axis=...)`

(Our math numbers axes $1 \ldots n$; NumPy uses $0 \ldots n-1$. Translate $k \mapsto k - 1$.)

Example: Elementwise vs Pairwise vs Reductions

```
import numpy as np

x = np.array([1.0, 2.0, 3.0])
y = np.array([10.0, 20.0, 30.0, 40.0])

E = x + 2.0              # elementwise unary
B = x[:, None] + y[None, :]    # pairwise via broadcasting -> outer sum
O = np.add.outer(x, y)         # explicit outer

M = np.arange(12).reshape(3, 4)
r0 = M.sum(axis=0)             # reduce over rows
r1 = M.sum(axis=1)             # reduce over cols
r1_keep = M.sum(axis=1, keepdims=True)

print("B shape:", B.shape, "O == B?", np.allclose(O, B))
print("r0/r1 shapes:", r0.shape, r1.shape, r1_keep.shape)
```

Intuition: Matrix products & contractions

```
import numpy as np
A = np.arange(6).reshape(2, 3)   # (2, 3)
B = np.arange(6).reshape(3, 2)   # (3, 2)
u = np.array([1., 2., 3.])       # (3,)

M = A @ B                        # (2, 2): sum of products along shared dim
d = A @ u                        # (2,):   matrix-vector
E = np.einsum('ij,jk->ik', A, B) # same as A @ B

print(M.shape, d.shape, np.allclose(M, E))
```

- Matrix multiply = **sum of products** along the shared axis.
- `einsum` expresses the same contraction with index labels.

Matrix Products and Tensor Contractions

- **Matrix multiply** `@`/`np.matmul`: for $A \in \mathbb{R}^{n \times k}$, $B \in \mathbb{R}^{k \times m}$, $(A@B)_{ij} = \sum_{\ell=1}^{k} A_{i\ell} B_{\ell j}$. With leading batch axes: $(\ldots, n, k)@(\ldots, k, m) \to (\ldots, n, m)$.
- **Dot product** `np.dot`: inner product for 1D, matrix product for 2D; for higher dims it contracts the last axis of the first with the second-to-last of the second.
- **General contractions**: `np.tensordot` specifies axes; `np.einsum` uses index notation.
- **Pairwise distances**: for $X \in \mathbb{R}^{M \times d}$, $Y \in \mathbb{R}^{N \times d}$,

$$\begin{aligned} D_{ij} &= \|X_{i\cdot} - Y_{j\cdot}\|_2 \\ &= \sqrt{\sum_{t=1}^{d} (X_{it} - Y_{jt})^2} \\ &= \sqrt{\|X_{i\cdot}\|_2^2 + \|Y_{j\cdot}\|_2^2 - 2\, X_{i\cdot} \cdot Y_{j\cdot}}. \end{aligned}$$

This is a *pairwise* mapping; it is not elementwise on X and Y.

Broadcasting for pairwise distance: expand_dims+tile vs broadcasting

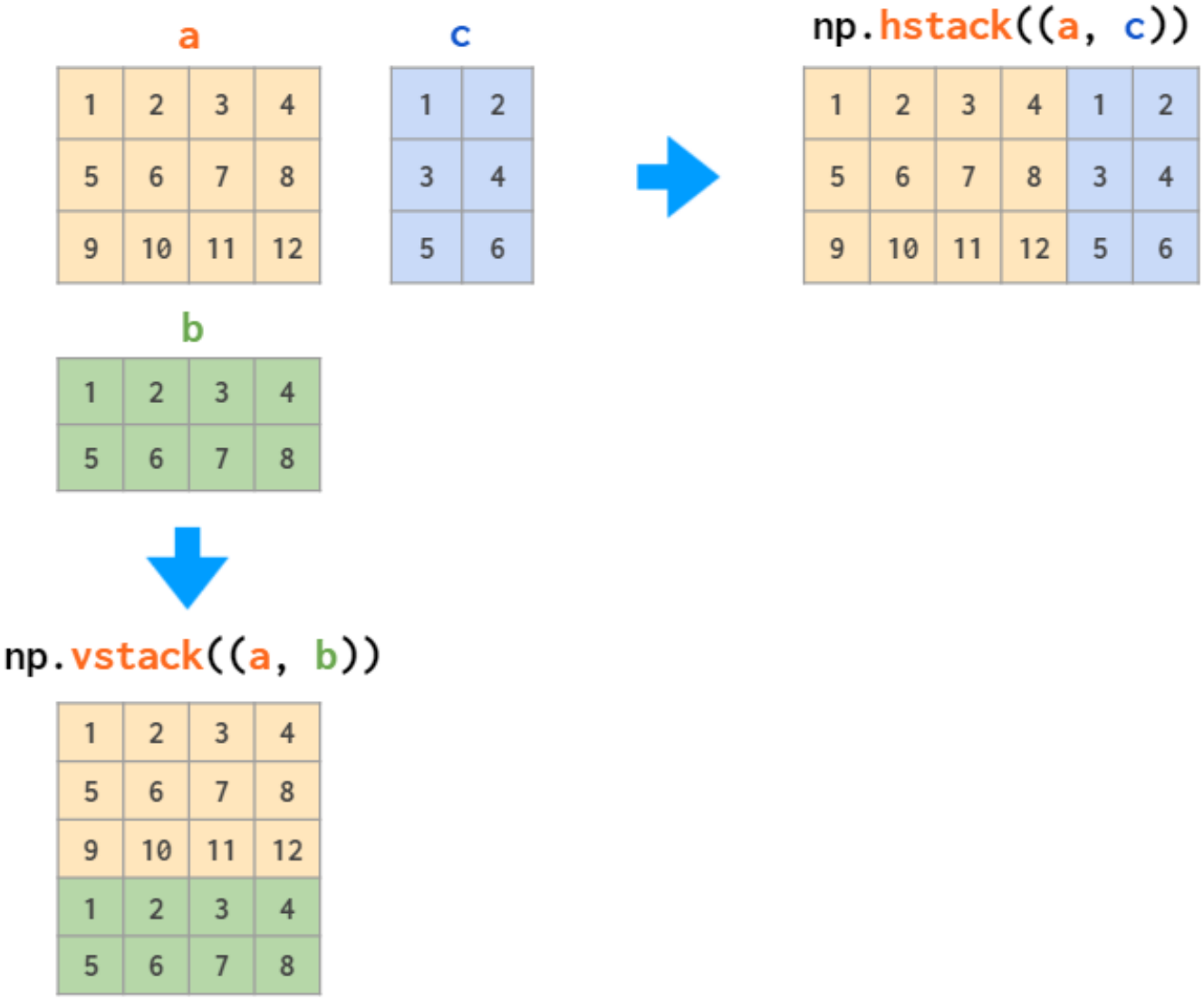
```
rng = np.random.default_rng(0)
samples = rng.random((15, 5))  # 15 points in R^5

# Without broadcasting: explicit tiling
expanded1 = np.expand_dims(samples, axis=1)                    # (15, 1, 5)
tile1     = np.tile(expanded1, (1, samples.shape[0], 1))       # (15, 15, 5)

expanded2 = np.expand_dims(samples, axis=0)                    # (1, 15, 5)
tile2     = np.tile(expanded2, (samples.shape[0], 1, 1))       # (15, 15, 5)

diff_nb   = tile2 - tile1                                      # (15, 15, 5)
dist_nb   = np.linalg.norm(diff_nb, axis=-1)                   # (15, 15)

# With broadcasting: no materialization of tiles
diff_bc = samples[:, np.newaxis, :] - samples[np.newaxis, :, :]  # (15, 15, 5)
dist_bc = np.linalg.norm(diff_bc, axis=-1)                        # (15, 15)

print("shapes:", diff_nb.shape, diff_bc.shape, dist_nb.shape, dist_bc.shape)
print("allclose?", np.allclose(dist_nb, dist_bc))
```

Both compute $D_{ij} = \|x_i - x_j\|_2$. Broadcasting avoids the $O(n^2 d)$ temporary created by `tile`.

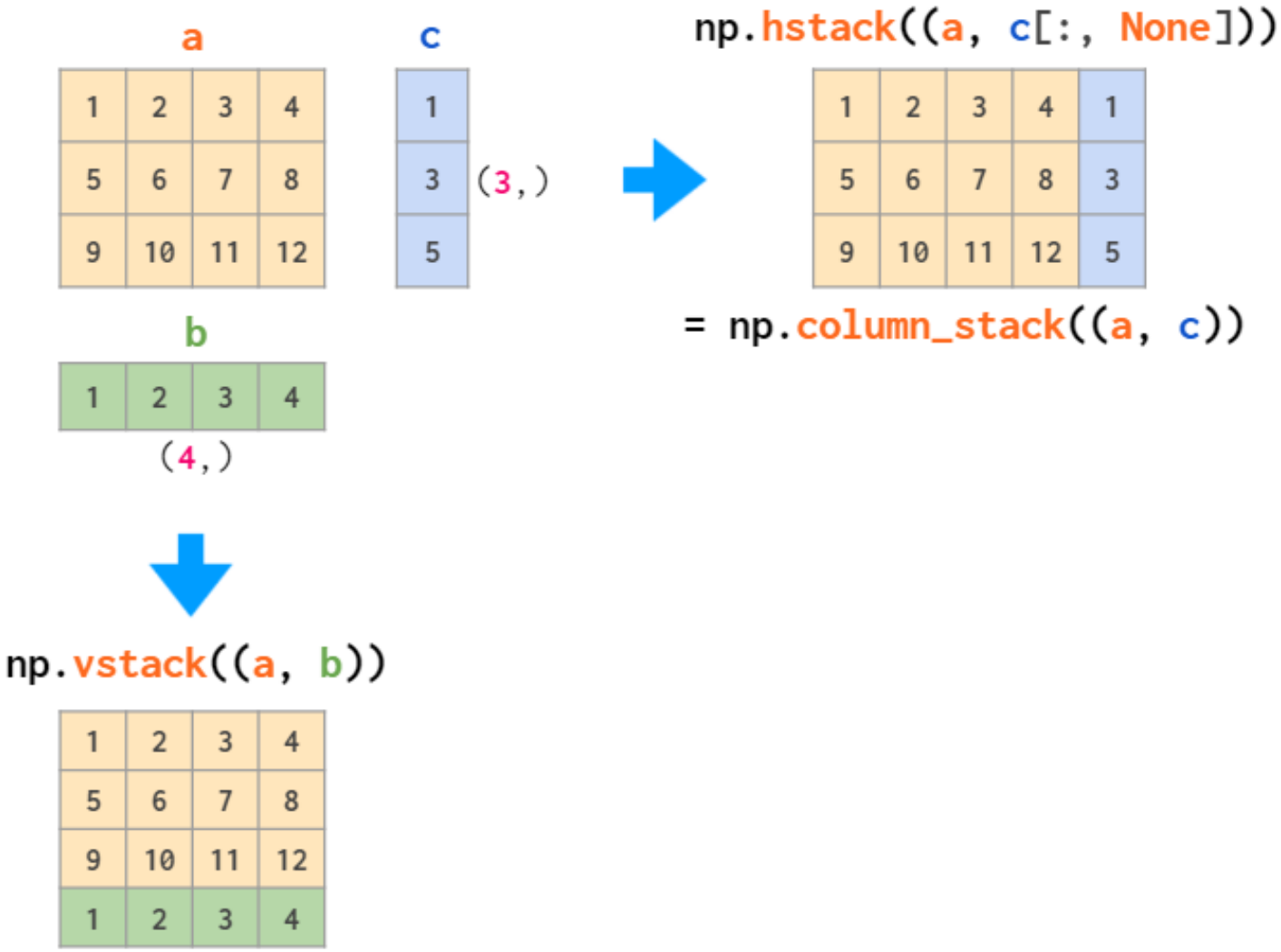
Example: Matmul, Dot, Tensordot, Einsum

```
import numpy as np

A = np.arange(6).reshape(2, 3)
B = np.arange(12).reshape(3, 4)

print("(A @ B).shape:", (A @ B).shape)

u = np.array([1.0, 2.0, 3.0])
v = np.array([4.0, 5.0, 6.0])
print("dot 1D:", np.dot(u, v))  # inner product

# Batched matmul: two samples
Ab = np.stack([A, A + 100], axis=0)   # shape (2, 2, 3)
Bb = np.stack([B, B + 100], axis=0)   # shape (2, 3, 4)
print("batched matmul shape:", (Ab @ Bb).shape)  # (2, 2, 4)

# Same as einsum
E = np.einsum('bij,bjk->bik', Ab, Bb)
print("einsum equals batched?", np.allclose(E, Ab @ Bb))

# Tensordot: contract last axis of A with first of B
TD = np.tensordot(A, B, axes=(1, 0))
print("tensordot == matmul?", np.allclose(TD, A @ B))
```

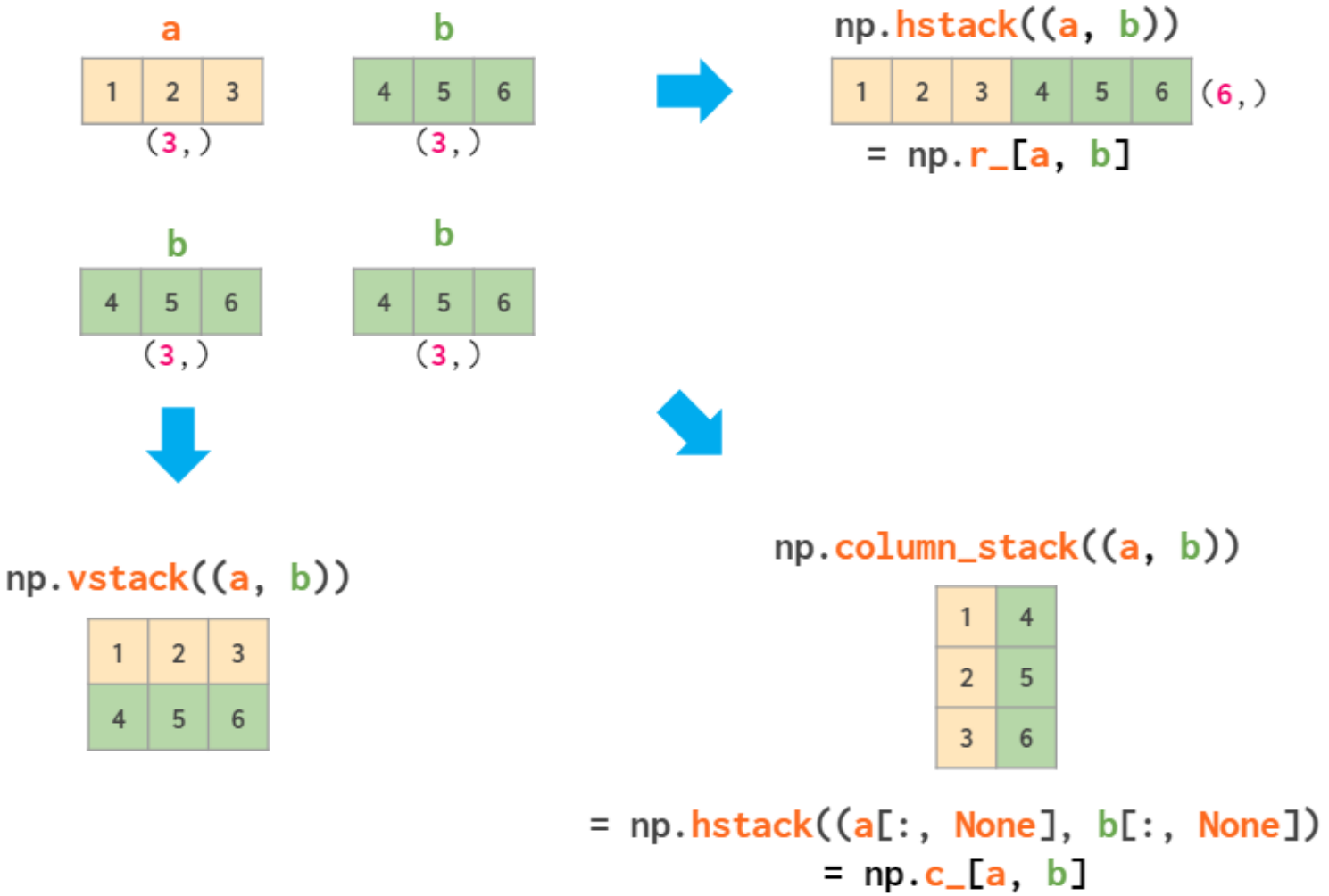
hstack vs vstack



column_stack for stacking a matrix and a vector



Staking two vectors



Intuition: vstack

```
import numpy as np
a = np.array([1, 2, 3])          # (3,)
b = np.array([10, 20, 30])        # (3,)
V = np.vstack([a, b])             # -> shape (2, 3)

A = np.arange(4).reshape(2, 2)    # (2, 2)
B = (A + 100)                     # (2, 2)
V2 = np.vstack([A, B])            # -> shape (4, 2)

print(V.shape, V2.shape)          # (2, 3) (4, 2)
```

- Stack **on top**: grow axis 0 (rows).
- 1D inputs become a **single row** before stacking.

Stacking with `vstack`: Formal semantics

Let $(A^{(r)})_{r=1}^{m}$ be arrays with shapes $s^{(r)} = (s_0^{(r)}, \ldots, s_{d_r-1}^{(r)})$ and dtype τ.
vstack (row-wise):

- Promote any 1D input x of shape $(N,)$ to a row $x^{\uparrow}$ of shape $(1, N)$ by $x^{\uparrow}[0, j] = x[j]$.
- After promotion, all inputs must have a common dimension $d \geq 2$ and agree on axes $1..d-1$.
- Let $\tilde{s}^{(r)}$ be the promoted shapes. The result Y has shape

$$\Big(\sum_{r=1}^{m} \tilde{s}_0^{(r)},\ \tilde{s}_1^{(1)}, \ldots, \tilde{s}_{d-1}^{(1)}\Big).$$

  With offsets $o_r = \sum_{p<r} \tilde{s}_0^{(p)}$,

$$Y[o_r + i, j_1, \ldots, j_{d-1}] \ = \ \tilde{A}^{(r)}[i, j_1, \ldots, j_{d-1}].$$

- Equivalent to `np.concatenate` along `axis=0` after 1D promotion.

Intuition: hstack

```
import numpy as np
a = np.array([1, 2, 3])          # (3,)
b = np.array([10, 20, 30])        # (3,)
H = np.hstack([a, b])             # -> shape (6,)

A = np.arange(4).reshape(2, 2)    # (2, 2)
B = (A + 100)                     # (2, 2)
H2 = np.hstack([A, B])            # -> shape (2, 4)

print(H.shape, H2.shape)          # (6,) (2, 4)
```

- Stack **side by side**: grow axis 1 (columns) for 2D and higher.
- For 1D inputs, it is vector concatenation.

Stacking with `hstack`: Formal semantics

Let $(A^{(r)})_{r=1}^{m}$ be arrays with shapes $s^{(r)} = (s_0^{(r)}, \ldots, s_{d_r-1}^{(r)})$ and dtype τ.
hstack (column-wise):

- Case 1: all inputs are 1D. Then it is plain concatenation of vectors. Result length $\sum_{r=1}^{m} s_0^{(r)}$ with mapping $Y[o_r + j] = A^{(r)}[j]$.
- Case 2: all inputs share a common $d \geq 2$ and agree on axes $\{0, 2, \ldots, d-1\}$. The result has shape

$$\left(s_0^{(1)},\ \sum_{r=1}^{m} s_1^{(r)},\ s_2^{(1)}, \ldots, s_{d-1}^{(1)}\right),$$

  and for offsets $o_r = \sum_{p<r} s_1^{(p)}$,

$$Y[i,\ o_r + j,\ j_2, \ldots, j_{d-1}] \ = \ A^{(r)}[i, j, j_2, \ldots, j_{d-1}].$$

- Equivalent to `np.concatenate` along `axis=1` for $d \geq 2$. Mixing 1D and $d \geq 2$ inputs is not allowed.

Example: vstack and hstack on 1D and 2D

```
a = np.array([1, 2, 3]) # 1D inputs
b = np.array([10, 20, 30])

V1 = np.vstack([a, b])   # (2, 3): 1D -> rows, then axis=0 concat
H1 = np.hstack([a, b])   # (6,): 1D concatenation

print("V1 shape:", V1.shape, "\n", V1)
print("H1 shape:", H1.shape, "\n", H1)

# 2D inputs
A = np.arange(6).reshape(2, 3)
B = np.arange(100, 106).reshape(2, 3)

V2 = np.vstack([A, B])   # (4, 3): stack rows
H2 = np.hstack([A, B])   # (2, 6): stack columns

print("V2 shape:", V2.shape)
print("H2 shape:", H2.shape)

# Equivalence to concatenate
V2_eq = np.concatenate([A, B], axis=0)
H2_eq = np.concatenate([A, B], axis=1)
print("V2 == concat0?", np.array_equal(V2, V2_eq))
print("H2 == concat1?", np.array_equal(H2, H2_eq))
```

What is batched matmul?

- For 2D matrices, `@` is the usual matrix product: $(M, K)@(K, N) \to (M, N)$.
- For higher rank arrays, `@` and `np.matmul` treat the **last two axes** as matrix dimensions and **all leading axes** as batch axes.
- Shape rule:

$$A \in \mathbb{R}^{\cdots \times M \times K}, \quad B \in \mathbb{R}^{\cdots \times K \times N} \implies Y = A@B \in \mathbb{R}^{\mathrm{broadcast}(\ldots,\ldots) \times M \times N}.$$

- Values (per batch index β):

$$Y[\beta, i, n] = \sum_{j=0}^{K-1} A[\beta, i, j]\, B[\beta, j, n].$$

- Leading batch axes must be equal or broadcastable.

Minimal examples: shapes

```
import numpy as np

# (B, M, K) @ (B, K, N) -> (B, M, N)
B, M, K, N = 2, 2, 3, 4
A = np.arange(B*M*K).reshape(B, M, K)
C = np.arange(B*K*N).reshape(B, K, N)
Y = A @ C
print(A.shape, C.shape, Y.shape)  # (2, 2, 3) (2, 3, 4) (2, 2, 4)

# (B, M, K) @ (K, N) -> (B, M, N)   (right broadcasts)
W = np.arange(K*N).reshape(K, N)
Y2 = A @ W
print(Y2.shape)                   # (2, 2, 4)

# (M, K) @ (B, K, N) -> (B, M, N)   (left broadcasts)
X = np.arange(M*K).reshape(M, K)
Y3 = X @ C
print(Y3.shape)                   # (2, 2, 4)
```

Example: Higher dims & error cases

```
import numpy as np

# Higher dims: vstack grows axis 0
C = np.zeros((3, 2, 2))
D = np.ones((4, 2, 2))
V3 = np.vstack([C, D])          # (7, 2, 2)
print("V3 shape:", V3.shape)

# Higher dims: hstack grows axis 1
E = np.zeros((5, 1, 2))
F = np.ones((5, 3, 2))
H3 = np.hstack([E, F])          # (5, 4, 2)
print("H3 shape:", H3.shape)

# vstack allows 1D with 2D (1D promoted to row)
V_mix = np.vstack([np.array([1, 2, 3]), np.array([[4, 5, 6]])])
print("V_mix shape:", V_mix.shape)  # (2, 3)

# hstack mixing 1D and 2D is not allowed
try:
    np.hstack([np.array([1, 2, 3]), np.array([[4, 5, 6]])])
except Exception as e:
    print("hstack mix error:", type(e).__name__)
```

Creating Arrays and Inspecting Properties

```
import numpy as np

a = np.array([1, 2, 3], dtype=np.int32)
b = np.arange(6).reshape(2, 3)
c = np.linspace(0.0, 1.0, num=5)

print(a.shape, a.dtype, a.ndim)
print(b.shape, b.size, b.T.shape)
print(c)
```

DType Rules and Casting

- Mixed Python types will upcast to a common `dtype`. Example: mixing integers and floats promotes to float.
- Strings in a numeric array will force an object or string `dtype`. Prefer explicit `dtype=` to avoid surprises.
- Use `astype` for explicit conversion when needed.

DType Pitfalls and Fixes

```
x = np.array([1, 2.5, 3])   # float dtype due to upcast
y = np.array([1, "2", 3])   # string/object dtype; avoid for math
z = np.array([1, 2, 3], dtype=np.float64)

print(x.dtype, y.dtype, z.dtype)
print(z.astype(np.int64))
```

Indexing and Slicing

- Basic slicing returns a view when possible; modifying the slice can modify the original.
- Integer and boolean indexing create copies.
- Be mindful of memory semantics when writing performance critical code.

Elementwise Ops and Reductions

- Elementwise: `+`, `-`, `*`, `/`, `np.multiply`, `np.sqrt`, `np.exp`, `np.log`.
- Reductions with `axis`: `sum`, `prod`, `min`, `max`, `argmin`, `argmax`, `mean`, `std`, `var`.
- Shapes matter for reduction results and broadcasting alignment.

Dot, Matmul, and Elementwise Multiply

```
A = np.arange(6).reshape(2, 3)
B = np.arange(6).reshape(3, 2)
x = np.array([1.0, 2.0, 3.0])

print(A @ B)              # matrix multiply
print(np.dot(A, x))       # matrix−vector
print(A * 2)              # elementwise scale
print(np.multiply(A, A)) # elementwise square
```

Linear Algebra Highlights

- `np.matmul` or `@` for matrix multiplication.
- `np.linalg.solve(A, b)` is preferred over `np.linalg.inv(A) @ b` for numerical stability.
- `np.linalg.eig` for eigen decomposition; `np.linalg.svd` for singular value decomposition.

Eigenvalues and Solving Linear Systems

```
A = np.array([[3.0, 1.0],
              [0.0, 2.0]])
w, V = np.linalg.eig(A)  # eigenvalues, eigenvectors
b = np.array([9.0, 4.0])
x = np.linalg.solve(A, b)
print(w)
print(x)
```

Broadcasting Rules

- Compare shapes from the trailing dimensions moving left.
- Dimensions are compatible if equal or one of them is 1.
- Missing leading dimensions are treated as 1.
- Result shape is the elementwise maximum along each dimension.

Broadcasting in Practice

```
u = np.array([1.0, 2.0, 3.0])          # shape (3,)
M = np.arange(12).reshape(4, 3) * 1.0 # shape (4, 3)

print(M + u)          # u broadcasts to (4, 3)
print((M.T + u).T)  # equivalent aligned form
```

Common Broadcasting Pitfalls

- Mismatched orientation: `op1 + op2` versus `op1 + op2.T` can change the result.
- Unintended expansion when a dimension equals 1; verify with `.shape`.
- Use `np.newaxis` to state intent and improve readability.

Pairwise Distances via Broadcasting

```
# Points in R^2
P = np.array([[0.0, 0.0],
              [1.0, 0.0],
              [0.0, 1.0]])

# Compute full distance matrix D_ij = ||P_i - P_j||
diff = P[:, np.newaxis, :] - P[np.newaxis, :, :]  # shape (3, 3, 2)
D = np.sqrt((diff ** 2).sum(axis=2))              # shape (3, 3)
print(D)
```

Why Vectorize

- Move work from Python loops to optimized C routines inside NumPy.
- Fewer interpreter overheads and better cache locality.
- Write clear, declarative code that maps to array algebra.

Cartesian Grid and Route-Style Distance Matrix

- Cartesian grid: construct `X` and `Y` by broadcasting one-dimensional coordinates.
- Distance matrix for locations with coordinates (x_i, y_i): $d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$.
- Broadcasting avoids nested Python loops and clarifies intent.

Cartesian Grid and Distance Matrix Code

```
xs = np.array([0.0, 1.0, 2.0, 3.0])
ys = np.array([0.0, 1.0, 2.0])

X = xs[np.newaxis, :]                    # shape (1, 4)
Y = ys[:, np.newaxis]                    # shape (3, 1)

# Manhattan distances on the grid as an example
D1 = np.abs(X - X.T) + np.abs(Y - Y.T)   # shape (3, 3) via broadcasti

# Euclidean distances between arbitrary points
P = np.stack([xs, np.linspace(0.0, 3.0, 4)], axis=1) # shape (4, 2)
diff = P[:, np.newaxis, :] - P[np.newaxis, :, :]
D2 = np.sqrt((diff ** 2).sum(axis=2))
print(D1.shape, D2.shape)
```

Debugging Friends

- Always check `arr.shape` and `arr.dtype` before heavy operations.
- Print small slices to validate broadcasting direction.
- Prefer `np.linalg.solve` over forming explicit inverses.
- Write vectorized expressions and avoid accidental copies when performance matters.

Key Takeaways

- Understand shapes and dtypes; they drive both correctness and speed.
- Embrace broadcasting with clear axis alignment using `np.newaxis` or `np.expand_dims`.
- Prefer vectorization and stable linear algebra routines.
- Verify with small examples, then scale up.